EXPLORATORY DATA ANALYSIS

CHAPTER 11

Visualizing Relationships

Seeing how variables vary together—without confusing association with explanation

Focus: scatterplots, overplotting, jitter, hexbin and 2D density, joint plots, pair plots, categorical–numerical comparisons, and nonlinear patterns.

Technical convention: Matplotlib object-oriented API; Seaborn axes-level functions receive explicit Axes objects.

Learning Objectives

- Choose a plot for numerical–numerical, categorical–numerical, and categorical–categorical relationships.
- Interpret scatterplots while considering direction, form, strength, clusters, and unusual observations.
- Recognize overplotting and select jitter, transparency, aggregation, or sampling appropriately.
- Use hexbin, 2D histograms, and contour/KDE plots to reveal dense bivariate structure.
- Use joint plots and pair plots for focused and multi-variable exploration.
- Visualize group differences and interactions without hiding group sizes or distributions.
- Distinguish association from causation and recognize confounding, selection, and aggregation effects.

Review of Earlier Chapters

Chapter 1 — What Is EDA?

EDA asks what the data contain, whether they can be trusted, and what patterns are present. A visible relationship is a descriptive finding, not a causal explanation.

Chapter 2 — Understanding the Dataset

Identify the observational unit, variable meanings, measurement scales, and granularity. Two columns may describe different levels (customer versus transaction), making a naive row-wise relationship invalid.

Chapter 3 — Data Quality Assessment

Duplicates, invalid values, missingness, inconsistent units, and data-entry errors can manufacture or distort relationships. Pairwise analysis also changes its denominator when rows are missing either variable.

Chapter 4 — Exploring Numerical Variables

Understand each numerical variable's center, spread, scale, skewness, and unusual values before interpreting a relationship.

Chapter 5 — Exploring Categorical Variables

Category frequencies, rare levels, and inconsistent labels matter when color, facets, or grouping encode categories.

Chapter 6 — Exploring Date and Time Variables

Time order, event gaps, and temporal coverage matter. Two variables may move together because both trend over time.

Chapter 7 — Descriptive Statistics

Use robust summaries and quantiles alongside plots. Means and standard deviations alone can hide subgroup structure and unusual values.

Chapter 8 — Understanding Distributions

Marginal distributions describe each variable separately. A bivariate plot adds how pairs of values occur together; it does not replace inspecting each marginal distribution.

Chapter 9 — Distribution Shape

Skewness, kurtosis, and quantiles help explain why a scatterplot may be dominated by extreme values or compressed central observations.

Chapter 10 — Visualizing Distributions

Histograms, KDEs, ECDFs, boxplots, violins, and strip plots describe one variable or compare its distribution across groups. Chapter 11 extends visual EDA to pairs of variables.

Review of Chapter 10 Assignment and Project

No assignment submission was included, so this is an expected-reasoning guide rather than grading.

- A histogram is useful for local concentration; an ECDF answers threshold and quantile questions without binning.
- KDE smoothness depends on bandwidth and can be misleading for small, discrete, or bounded data.
- Boxplots summarize median and quartiles but hide multimodality; violin plots add a smoothed shape estimate.
- Group comparisons should use common bins and scales where direct comparison is intended.
- The project should return reusable figure/axes objects and explain when each visualization is unsuitable.

Expected project learning: select visual encodings according to the question, data type, sample size, and limitations.

Preview of This Chapter

Relationships require us to examine pairs of observations rather than separate columns. We will begin with scatterplots, then handle dense data, grouped comparisons, and categorical combinations. Throughout, we will ask what the plot establishes—and what it cannot establish.

1. What Is a Relationship Between Variables?

A relationship describes how the values or categories of one variable vary with another. It may be linear, nonlinear, monotonic, non-monotonic, clustered, conditional on a third variable, or absent in the observed data.

Pair of variable types	Useful starting plots
Numerical × numerical	Scatterplot, hexbin, 2D histogram, 2D KDE
Categorical × numerical	Strip/swarm, boxplot, violin, point/range plot
Categorical × categorical	Count/proportion heatmap, mosaic-style display, grouped bars

Always inspect each variable's marginal distribution too. A relationship plot can hide skew, rare categories, or outliers that strongly influence its appearance.

2. Scatterplots: The Fundamental Bivariate Plot

A scatterplot places one observation at coordinate (x, y). It is usually the first choice for two numerical variables.

- Direction: do larger x values tend to occur with larger y values (positive) or smaller y values (negative)?
- Form: is the pattern approximately linear, curved, threshold-like, periodic, or otherwise structured?
- Strength: how tightly do observations follow a pattern?
- Structure: are there clusters, gaps, subgroups, or changing spread?
- Unusual observations: are there points far from the main cloud or influential x/y extremes?

```
fig, ax = plt.subplots(figsize=(7, 5))
sns.scatterplot(
    data=df,
    x="study_hours",
    y="exam_score",
    ax=ax
)
ax.set(
    title="Study hours and exam score",
    xlabel="Study hours",
    ylabel="Exam score"
)
fig.tight_layout()
```

A positive association does not mean every observation follows the trend. Describe the overall pattern and exceptions, not an imagined deterministic rule.

3. Overplotting: When Points Hide Points

Overplotting occurs when many observations occupy the same or nearby coordinates. A dense area may look like one solid blob, and repeated values can conceal how many records overlap.

Transparency and point size

```
fig, ax = plt.subplots(figsize=(7, 5))
sns.scatterplot(
    data=df,
    x="income",
    y="spending",
    alpha=0.25,
    s=22,
    edgecolor=None,
    ax=ax
)
ax.set(xlabel="Income", ylabel="Spending")
fig.tight_layout()
```

Transparency (alpha) allows dense regions to appear darker. It is useful but not a precise count scale; overlapping opacity depends on rendering and settings.

Jitter for discrete or rounded values

Jitter adds a small random horizontal or vertical displacement so coincident points can be seen. It is a display adjustment, not a change to the source data. State or document jitter when exact coordinates matter.

```
fig, ax = plt.subplots(figsize=(7, 5))
sns.stripplot(
    data=df,
    x="rating",
    y="waiting_minutes",
    jitter=0.22,
    alpha=0.45,
```

```
    ax=ax
)
ax.set(xlabel="Rating", ylabel="Waiting time (minutes)")
fig.tight_layout()
```

Sampling

For very large datasets, a random sample can reveal broad structure more clearly. Sampling may hide rare but important patterns; disclose the sampling method, size, and seed, and retain full-data summaries where possible.

4. Hexbin and Two-Dimensional Histograms

When point clouds are too dense, aggregate observations into two-dimensional cells. Hexbin uses hexagonal cells; a 2D histogram uses rectangular bins. Color encodes the number of observations in each cell.

```
fig, ax = plt.subplots(figsize=(7, 5))
hb = ax.hexbin(
    df["income"],
    df["spending"],
    gridsize=35,
    mincnt=1,
    cmap="viridis"
)
fig.colorbar(hb, ax=ax, label="Number of observations")
ax.set(
    title="Joint density by hexagonal bins",
    xlabel="Income",
    ylabel="Spending"
)
fig.tight_layout()
```

Cell size and grid placement affect the apparent pattern. Try defensible resolutions and avoid treating cell boundaries as meaningful data boundaries. A colorbar must explain the encoded quantity.

```
fig, ax = plt.subplots(figsize=(7, 5))
h2 = ax.hist2d(
    df["income"],
    df["spending"],
    bins=35,
    cmap="viridis"
)
fig.colorbar(h2[3], ax=ax, label="Count")
ax.set(xlabel="Income", ylabel="Spending")
fig.tight_layout()
```

5. Two-Dimensional Density and Contours

A bivariate KDE estimates a smooth joint density. Filled contours or contour lines can reveal dense regions and broad structure, but smoothing can blur clusters or create apparent bridges between them.

```
fig, ax = plt.subplots(figsize=(7, 5))
sns.kdeplot(
    data=df,
    x="income",
    y="spending",
    fill=True,
    levels=12,
    thresh=0.05,
    ax=ax
)
ax.set(
    title="Estimated bivariate density",
    xlabel="Income",
    ylabel="Spending"
)
fig.tight_layout()
```

- KDE depends on bandwidth in both dimensions and on variable scales.
- Strongly different units or scales can affect smoothing; consider whether rescaling is analytically justified and document it.
- Bivariate KDE can be unreliable for small samples, discrete variables, or strongly bounded data.
- Contours represent estimated density levels, not causal boundaries or confidence regions.

6. Marginal and Joint Views

A joint plot combines a bivariate display with marginal distributions of x and y. This helps connect the relationship to each variable's distribution.

```
g = sns.jointplot(
    data=df,
    x="income",
    y="spending",
    kind="scatter",
    height=6
)
```

Seaborn's jointplot is figure-level and returns a JointGrid, not a single Axes. It is convenient for exploration. For publication or highly customized layouts, construct the axes explicitly with Matplotlib.

7. Pair Plots for Several Numerical Variables

A pair plot creates a matrix of pairwise plots. Diagonal cells often show each variable's distribution; off-diagonal cells show pairwise relationships. It is useful for an initial scan of a modest number of numerical variables.

```
plot_df = df[
    ["age", "income", "spending", "visits"]
].dropna()

g = sns.pairplot(
    plot_df,
    diag_kind="hist",
    corner=True
)
```

Pair plots scale poorly as the number of variables grows: the number of pairs increases rapidly, labels become crowded, and irrelevant combinations distract. Select variables with a clear analytical reason and consider sampling large datasets.

8. Numerical Relationships With Groups

Color or facets can reveal whether a relationship differs across groups. A pooled trend may hide different subgroup patterns.

```
fig, ax = plt.subplots(figsize=(7, 5))
sns.scatterplot(
    data=df,
    x="study_hours",
    y="exam_score",
    hue="school_type",
    style="school_type",
    alpha=0.7,
    ax=ax
)
ax.set(
    title="Study time and score by school type",
    xlabel="Study hours",
    ylabel="Exam score"
)
ax.legend(title="School type")
fig.tight_layout()
```

Use a restrained number of groups and a readable legend. If many categories exist, use small multiples or select a meaningful subset rather than encoding dozens of colors.

9. Categorical–Numerical Relationships

When one variable is categorical and the other numerical, compare the numerical distribution across categories. A single mean per group can conceal skewness, spread, and outliers.

```
fig, ax = plt.subplots(figsize=(8, 4))
sns.boxplot(
    data=df,
    x="department",
    y="salary",
    ax=ax
)
ax.set(
    title="Salary distribution by department",
    xlabel="Department",
    ylabel="Salary"
)
fig.tight_layout()
```

A violin plot can show smoothed distribution shape; a strip or swarm plot shows observations. Combining a boxplot with a transparent strip plot can provide both summary and raw-data context, but may become cluttered.

```
fig, ax = plt.subplots(figsize=(8, 4))
sns.violinplot(
    data=df,
    x="department",
    y="salary",
    inner="quart",
    cut=0,
    ax=ax
)
sns.stripplot(
    data=df,
    x="department",
    y="salary",
    color="black",
    alpha=0.25,
    size=2,
    jitter=0.18,
    ax=ax
)
ax.set(xlabel="Department", ylabel="Salary")
fig.tight_layout()
```

Category order should be intentional—natural order, business order, or a clearly stated sorting rule. Avoid silently sorting categories by a statistic if that changes the analytical question.

10. Categorical–Categorical Relationships

For two categorical variables, contingency tables are the quantitative foundation. Visualizations can show counts or proportions across combinations.

```
ct = pd.crosstab(
    df["channel"],
    df["subscription"],
    normalize="index"
)

fig, ax = plt.subplots(figsize=(7, 4))
sns.heatmap(
    ct,
    annot=True,
    fmt=".1%",
    cmap="Blues",
    vmin=0,
    vmax=1,
    ax=ax
)
ax.set(
    title="Subscription proportions within channel",
    xlabel="Subscription",
    ylabel="Channel"
)
fig.tight_layout()
```

Row normalization answers: within each channel, what proportion belongs to each subscription category? Column normalization answers a different question. Always state the denominator. A count heatmap and a proportion heatmap communicate different things.

11. Nonlinear and Non-Monotonic Patterns

A scatterplot may show a relationship that is not well described by a straight line.

- Curved: y may increase and then level off, or follow a U-shaped pattern.
- Threshold: the relationship changes after a particular x value.
- Heteroscedastic: vertical spread changes as x changes.
- Periodic: values rise and fall in cycles.
- Clustered: different subpopulations occupy separate regions.
- Non-monotonic: y does not consistently increase or decrease as x increases.

```
fig, ax = plt.subplots(figsize=(7, 5))
sns.regplot(
    data=df,
    x="temperature",
    y="energy_use",
    lowess=True,
    scatter_kws={"alpha": 0.35, "s": 20},
    line_kws={"color": "crimson"},
    ax=ax
)
ax.set(xlabel="Temperature", ylabel="Energy use")
fig.tight_layout()
```

LOWESS adds a locally smoothed descriptive trend. It is not a causal model, and its shape depends on smoothing choices. Inspect the raw points; a smooth line can conceal sparse regions or subgroup mixtures.

12. Correlation Is Not the Whole Relationship

Correlation coefficients summarize selected aspects of association. Pearson correlation measures linear association; rank-based measures such as Spearman correlation measure monotonic association. Neither replaces a plot.

- A strong nonlinear relationship can have a weak Pearson correlation.
- A few influential points can change correlation substantially.
- A mixture of groups can create or reverse an overall pattern.
- Zero correlation does not imply independence in general.
- Association does not establish causation.

Quantitative association measures receive a dedicated treatment in Chapter 13. This chapter uses plots to discover form, clusters, and exceptions before choosing a numerical measure.

13. Confounding, Simpson's Paradox, and Aggregation

A third variable can influence both variables being plotted. The apparent relationship between x and y may change after separating observations by that third variable. This is one reason to inspect meaningful groups.

Simpson's paradox describes situations where a trend present within several groups reverses or changes when the groups are combined. Different group sizes and group-level differences can produce this effect.

- Compare pooled and stratified plots when a relevant grouping variable exists.

- Do not stratify indiscriminately; choose groups based on domain knowledge and the data-generating process.
- Aggregated averages can hide within-group variation and produce ecological fallacies.
- A group-level association does not automatically apply to individuals.

A plot can reveal possible confounding, but cannot by itself determine causal structure. Causal claims require design knowledge and appropriate methods beyond descriptive EDA.

14. Missing Data and Pairwise Comparisons

A scatterplot generally includes only rows with valid values for both plotted variables. Its sample size may differ from the dataset size and from other pairwise plots.

```
pair = df[["income", "spending"]].dropna()

print("Rows in dataset:", len(df))
print("Complete pairs:", len(pair))
print("Rows excluded:", len(df) - len(pair))
```

If missingness depends on either variable or a third factor, the displayed relationship may not represent the full population. Report complete-pair n and consider whether excluded records differ systematically.

15. A Reusable Pairwise Plot Helper

This helper provides a compact scatterplot with explicit validation and returns the figure and axes for further customization.

```
def plot_relationship(data, x, y, hue=None, figsize=(7, 5)):
    columns = [x, y] + ([hue] if hue is not None else [])
    plot_data = data[columns].copy()
    plot_data[x] = pd.to_numeric(
        plot_data[x], errors="coerce"
    )
    plot_data[y] = pd.to_numeric(
        plot_data[y], errors="coerce"
    )
    plot_data = plot_data.dropna(subset=[x, y])

    if plot_data.empty:
        raise ValueError("No complete numeric pairs to plot.")

    fig, ax = plt.subplots(figsize=figsize)
    sns.scatterplot(
        data=plot_data,
        x=x,
        y=y,
        hue=hue,
        alpha=0.55,
        ax=ax
    )
    ax.set(
        title=f"{y} versus {x}",
        xlabel=x,
        ylabel=y
    )
    fig.tight_layout()
    return fig, ax, plot_data
```

The returned plot_data makes the actual plotted sample inspectable. A professional implementation should also record dropped-row counts, support optional sampling, and allow display settings to be configured.

16. Visual Design Checklist

- State the observational unit and sample size where useful.
- Use axis labels with units and a descriptive title.
- Check whether a log scale is justified; label it clearly and explain zero/negative handling.
- Use transparency or aggregation when points overlap heavily.
- Do not use jitter to imply that modified coordinates are observed values.
- Use color and marker shape consistently and accessibly.
- Keep group scales comparable when making direct comparisons.
- Show marginal distributions when they materially affect interpretation.
- Identify missing-pair exclusions and any sampling.
- Describe patterns cautiously and separate observations from possible explanations.

17. Common Mistakes

- Treating every cloud of points as linear.
- Assuming a trend line explains every observation.

- Ignoring overplotting and interpreting dense points as a single cluster.
- Using jitter without recognizing that it changes displayed coordinates.
- Using KDE contours as if they were exact boundaries.
- Creating huge pair plots with no analytical focus.
- Comparing groups with different axes or incompatible scales.
- Reporting only group means when distributions differ in shape or spread.
- Using row percentages without explaining the denominator.
- Ignoring missing-pair exclusions.
- Interpreting a pooled association without checking meaningful subgroups.
- Claiming causation from observational association.

Chapter Summary

- Scatterplots are the starting point for two numerical variables; inspect direction, form, strength, clusters, and unusual points.
- Transparency, jitter, sampling, hexbin, and 2D histograms help with overplotting.
- Bivariate KDE and contours show smoothed density but depend on bandwidth and data suitability.
- Joint plots add marginal distributions; pair plots scan several variables but can become unwieldy.
- Use boxplots, violins, and point plots for categorical–numerical comparisons.
- Use contingency tables and normalized heatmaps for categorical–categorical comparisons.
- Nonlinear patterns, group mixtures, and confounding can make simple summaries misleading.
- Report pairwise sample sizes and avoid causal claims.

Assignment

Part A — Conceptual Questions

- What five features should you inspect in a scatterplot?
- Explain overplotting and compare transparency, jitter, sampling, and hexbin.
- When is a 2D KDE useful, and what are its limitations?
- What does a joint plot add to a scatterplot?
- Why can pair plots become impractical as the number of variables grows?
- How would you visualize a numerical variable across categories without hiding its spread?
- What is the denominator in a row-normalized contingency table?
- Explain how a third variable can change an apparent relationship.
- What is Simpson's paradox in descriptive terms?
- Why does a scatterplot not establish causation?

Part B — Numerical–Numerical

Choose two numerical variables from a dataset you understand.

- Create a scatterplot with appropriate labels and units.
- Inspect overplotting and create a second view using transparency or hexbin.
- Add a LOWESS descriptive smoother only if it helps reveal structure.
- Report complete-pair n and describe direction, form, spread, clusters, and unusual points.
- Inspect whether a third variable changes the pattern.

Part C — Mixed Variable Types

- Choose one numerical and one categorical variable; compare distributions with boxplot and violin or strip plot.
- Choose two categorical variables; create a crosstab and row-normalized heatmap.
- Explain the denominator and show group sample sizes.
- Compare pooled and subgroup patterns where a meaningful third variable exists.

Part D — Critical Interpretation

Find a relationship that looks different when plotted as a pooled sample versus separate groups. Describe the two views, explain how group composition could affect the pooled pattern, and state what additional evidence would be needed before making a causal claim.

Coding Project — Relationship Visualization Toolkit

Extend the cumulative EDA master notebook with a reusable relationship-visualization section. Reuse the distribution-plotting and summary components from earlier chapters.

- Implement plot_relationship() for numerical pairs, with validation and complete-pair counts.
- Add optional hue grouping and a documented sampling option for large datasets.
- Create hexbin or 2D histogram plots for dense data.
- Create a categorical–numerical comparison function using boxplots and optional raw points.
- Create a categorical–categorical proportion heatmap with configurable normalization direction.
- Add a pair-plot workflow that requires an explicit list of numerical columns.
- Include at least one nonlinear or subgroup example and explain the visual evidence.
- Document plot limitations, missingness, sample sizes, and the difference between association and causation.

Expected reusable components: plot_relationship(), plot_hexbin_relationship(), plot_categorical_numeric(), and plot_categorical_heatmap(). Return figure/axes objects where practical and keep the original data unchanged.

Deliverables: updated cumulative EDA master notebook, reusable functions, saved visualizations, and a concise relationship-analysis report.

Preview of Next Chapter

Chapter 12 — Multivariate Visualization

We'll extend beyond pairs of variables and examine several dimensions together. Topics include visual encodings, color/size/shape, faceting, parallel coordinates, heatmaps, correlation matrices as visual summaries, and dimensionality challenges. We'll focus on avoiding clutter and preserving interpretability when multiple variables interact.